COMPUTER SCIENCE + COMPUTER ENGINEERING

Module 19 Networks, storage and data services

Individual module / Expanded Volume 04

Four lessons with original foundations, expanded study, worked examples, laboratory checkpoints and quizzes.

Lessons 19.1–19.4 · 20 questions · One integration project

Package 05 · Expanded Volume 04 · September 28, 2026
Self-study teaching edition. Completion requires independent implementation, testing and review.

Module 19: Networks, storage and data services

Original prerequisites: Modules 15 and 17-18.

Integration goal: Local message and data service. Build a loopback or in-process protocol harness and a relational inventory operation. State exactly which behavior belongs to transport, parsing and persistence.

19.1 — Network layers, latency and transport semantics

Learning objective: Choose transport behavior from application requirements.

Original foundation

Networks exchange data through layered protocols. IP provides packet delivery without guaranteeing arrival or order. TCP presents a reliable ordered byte stream, not a stream of application messages. UDP preserves datagram boundaries but does not itself supply delivery, ordering or congestion-control behavior. Applications still need explicit protocols. Latency, jitter, loss and bandwidth describe different constraints. A high-bandwidth link can still have long round-trip delay. Reliable delivery can involve waiting for missing information, which affects real-time design. Use localhost or an authorized lab network for exercises and keep authentication and encryption decisions separate from transport assumptions.

Expanded study

Choose a transport property, not a slogan

TCP gives the application an ordered byte stream, not a matching receive for each send call. UDP datagrams retain boundaries but do not by themselves supply the application's reliability, ordering, congestion response or security policies. A high-capacity link can still have high delay. Keep bandwidth, latency, jitter and loss separate when reasoning about game messages. [R17]

Make a message boundary explicit

The lab uses a four-byte unsigned big-endian payload length followed by that many bytes. Zero and lengths above 4096 are rejected. A receiver first collects the header, validates it, then collects the bounded body. An advertised length of 0xffffffff is an error, not an invitation to allocate almost four gigabytes. The incremental decoder can emit several messages from one chunk and retain an incomplete tail for the next chunk.

Test segmentation without relying on the operating system

The demo passes deliberately split byte chunks directly to FrameDecoder. Its tests try every split point of two concatenated frames. A localhost example also makes several send calls, but the OS is free to coalesce or split those calls differently; that experiment alone does not force a particular receive pattern. Both the parser-level test and the socket integration test are needed for their different evidence. [P09]

State the input and buffering budgets

One incremental feed accepts at most 8192 bytes, and retained incomplete frame data is bounded by the 4096-byte payload contract plus header. The number of returned message objects is also bounded by that accepted chunk, but Python object overhead remains separate from wire-byte counts. A clean end-of-stream is valid only when no partial frame remains. Once a framing error occurs, the reference decoder is unusable; it does not guess where a new header might start.

Worked reasoning

The frame for the ASCII bytes abc is 00 00 00 03 61 62 63 in hexadecimal. Two frames can be concatenated without a separator because each prefix gives the next body length. Receiving only 00 00 and then the rest is valid fragmentation; reaching EOF after a complete header and only one of three payload bytes is truncation. The socket reader distinguishes EOF from deadline expiration and invalid advertised length.

Runnable example

File: `python/examples/lesson_19_1.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 19.1 --build-dir build
```

```
from python.reference.lab_19_1 import FrameDecoder,encode_frame
wire=encode_frame(b"one")+encode_frame(b"two")
decoder=FrameDecoder();result=[]
for chunk in [wire[:2],wire[2:6],wire[6:]]:
    result.extend(decoder.feed(chunk))
decoder.finish()
print(result)
```

Expected output from the supplied reference in the validated environment:

```
[b'one', b'two']
```

Practical checkpoint

Create a localhost client/server exchange with a length-prefixed message. Pass: split one message across several writes and combine several messages in one write; the receiver reconstructs them correctly.

Expanded contract: Frames contain a 4-byte network-order length and 1..4096 payload bytes. FrameDecoder.feed accepts bytes chunks <=8192 and emits complete bodies; finish rejects a partial tail. recv_frame reads one frame by an absolute monotonic deadline. Its socket timeout is adjusted as part of the operation.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `python/student/lab_19_1.py`. Use `python3 check_lab.py 19.1 --student` after implementation.

Independent transfer

Your server receives two commands in one recv call. Is TCP broken?

Quiz — answer before opening the key

1. **Does TCP preserve application message boundaries?**
2. **Does high bandwidth imply low latency?**
3. **What behavior must an application add when using UDP?**
4. **What is the seven-byte wire frame for abc?**
5. **Why close a decoder after an invalid length rather than scanning for another apparent prefix?**

Assigned study

R17: TCP service and interface semantics; R16: networking context.

Expansion source map: R17, P09. See the source directory for the exact documents and version notes.

19.2 — Protocol design, serialization and timeouts

Learning objective: Treat every received byte as untrusted structured input.

Original foundation

A protocol defines message types, fields, ranges, versioning and permitted sequences. Serialization must specify byte order and numeric representation. Validate lengths before allocation and cap total resource use. A checksum can detect some corruption but is not equivalent to cryptographic authentication. Timeouts handle missing progress, not certainty about remote state. A timed-out operation may have executed remotely even if its response was lost. Request IDs and idempotent operations help manage retries. Compatibility is a design decision: unknown fields and versions should be accepted, ignored or rejected according to a documented rule.

Expanded study

Validate a complete request contract

Version 1 has exactly four fields: version, id, operation and values. The operation is sum_squares; values is a list of 1..256 integers in 0..1000; IDs contain 1..32 ASCII letters, digits, underscore or hyphen. Booleans are deliberately rejected as numeric values even though Python bool is related to int. Unknown fields and versions are errors. These are chosen protocol rules, not claims that every game should use this schema.

Make parsing ambiguity visible

The parser rejects invalid UTF-8, duplicate JSON field names, non-finite constants and inputs outside the byte limit. It catches excessive nesting/parser recursion as an invalid request. A syntactically valid JSON object is not automatically a valid job. In particular, a client cannot choose an executable, path, arbitrary expression or number of worker threads. Those capabilities are absent from the protocol rather than filtered through a shell later.

Use one absolute deadline

Setting a fresh full timeout for every fragment can let a peer extend total wait time by trickling bytes. The reference calculates remaining=deadline-monotonic_now before each read, so the entire frame shares one budget. A timeout after submission still does not establish whether work committed. The gateway reports UNKNOWN_OUTCOME and tells the caller to retain the same ID and payload when retrying. [P09]

Bind idempotency identity to the work

The store compares a canonical validated request with the stored request for that ID. A repeated ID with identical work may reuse the result; the same ID with different values is rejected. Deduplication has a retention scope: deleting the record can remove protection for a future retry. The reference caps history and rejects new unique IDs when full rather than silently evicting old evidence. This bounds the exercise while keeping its stated replay guarantee simple.

Worked reasoning

Request {version:1,id:job-1,operation:sum_squares,values:[2,3]} represents result 13. Reordering JSON object fields should not change the canonical request. Replacing 2 with true must fail type validation. Retrying job-1 with [2,3] after a lost response should not create another durable record. Retrying job-1 with [2,4] is a conflict, not an update to the original job.

Runnable example

File: `python/examples/lesson_19_2.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 19.2 --build-dir build
```

```
from python.reference.lab_19_2 import parse_request,canonical_request
raw=b'{"version":1,"id":"job-1","operation":"sum_squares","values":[2,3]}'
print(canonical_request(parse_request(raw)))
try:
    parse_request(raw.replace(b'[2,3]',b'[true,3]'))
except ValueError:
    print("boolean value rejected")
```

Expected output from the supplied reference in the validated environment:

```
{"id":"job-1","operation":"sum_squares","values":[2,3],"version":1}
boolean value rejected
```

Practical checkpoint

Add versioned framing, maximum sizes, timeout handling and request IDs to the lab protocol. Pass: reject oversized and truncated messages before dangerous allocation and test duplicate delivery.

Expanded contract: Python parse_request(bytes) accepts only the exact bounded version-1 schema. canonical_request validates again and emits deterministic sorted compact JSON. IDs do not authorize actions; the local gateway intentionally supplies neither public authentication nor encryption.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `python/student/lab_19_2.py`. Use `python3 check_lab.py 19.2 --student` after implementation.

Independent transfer

The client timed out and invents a new ID for the retry. What guarantee has it lost?

Quiz — answer before opening the key

1. **Does a timeout prove the remote operation never happened?**
2. **Why validate a length before allocating its buffer?**
3. **What does idempotence mean?**
4. **Why reject bool values explicitly in a Python integer schema?**
5. **What does the reference do when result history is full?**

Assigned study

R17: transport failure context; R18: transaction and data-integrity foundations.

Expansion source map: R17, P09, P10, P12. See the source directory for the exact documents and version notes.

19.3 — Relational modeling, SQL and indexes

Learning objective: Represent game data with constraints that survive application mistakes.

Original foundation

A relational schema defines tables, keys and relationships. A primary key identifies a row; a foreign key constrains references between tables. Normalization reduces certain redundancy and update anomalies, while deliberate denormalization trades simpler reads for additional consistency work. SQL queries express desired results, but their execution plans determine physical work. An index can accelerate suitable lookups while consuming storage and adding write cost. Parameterized queries separate data from executable query structure. Do not build SQL by concatenating untrusted text, and do not assume an index helps every query.

Expanded study

Put relationships in the schema

The lab separates players from inventory. A primary key identifies a player. An inventory row references that player and has a composite key (player_id,item_id), preventing two aggregate rows for the same player/item pair. A quantity constraint rejects negative or out-of-contract amounts even if application code makes a mistake. Schema constraints complement validation; neither replaces the other. [R18]

Know which database you are actually running

The executable exercises use SQLite through Python's standard library so no server installation is required. Each connection explicitly enables foreign-key enforcement and uses explicit BEGIN/COMMIT control. PostgreSQL remains the assigned relational reading, and a separate SQL companion supports a PostgreSQL exercise. Do not copy SQLite PRAGMAs, locking behavior or placeholder syntax into a PostgreSQL report and call them identical. [P10, P11, P12]

Separate data from query syntax

All values in the reference are passed as parameters. An unusual player name containing quotes remains one data value; it is not concatenated into a command. Parameterization does not validate gameplay rules, and it does not turn arbitrary user-selected table names into safe values. The schema and supported operations stay under application control. A query should request only the fields needed for its purpose. [R18, P10]

Explain an index using the query it supports

The composite inventory key supports a lookup whose leading field is player_id. A separate item_id index can help item-focused queries, depending on selectivity and the optimizer. Indexes consume storage and maintenance work. Inspect EXPLAIN output and verify the query result before and after adding an index. An optimizer may rationally choose a scan for a tiny table, so seeing no index use is not automatically a database defect. [R18]

Worked reasoning

Seed player A with 20 ore and B with 3 ore. A query ordered by player_id returns [(A,20),(B,3)]. An inventory insert for a nonexistent player fails the foreign-key constraint. A second (A,ore) aggregate row fails uniqueness. Updating quantity to -1 fails the domain constraint. These tests deliberately exercise the database directly rather than relying only on an application helper that may reject the input first.

Runnable example

File: `python/examples/lesson_19_3.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 19.3 --build-dir build
```

```
from python.reference.lab_19_3 import connect_store,seed_inventory
import sqlite3

db=connect_store()
try:
    seed_inventory(db)
    print(db.execute("SELECT player_id,quantity FROM inventory ORDER BY player_id").fetchall())
    try:
        db.execute("INSERT INTO inventory VALUES (?,?,?)",("missing","ore",1))
    except sqlite3.IntegrityError:
        print("foreign key rejected")
finally:
    db.close()
```

Expected output from the supplied reference in the validated environment:

```
[('A', 20), ('B', 3)]
foreign key rejected
```

Practical checkpoint

Create a small schema with primary, foreign and uniqueness constraints. Use parameterized queries and inspect one indexed query plan. Pass: demonstrate rejected invalid references and compare a query before/after indexing.

Expanded contract: Python connect_store creates players, inventory and operations tables and enables foreign keys. seed_inventory atomically creates A/B and ore quantities 20/3. Explicit SQL controls transactions. A connection belongs to its thread and must be closed by its owner.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `python/student/lab_19_3.py`. Use `python3 check_lab.py 19.3 --student` after implementation.

Independent transfer

A tiny-table query ignores a newly created index. Must the learner force index use to pass?

Quiz — answer before opening the key

1. **What does a foreign key protect?**
2. **Why can indexes slow writes?**
3. **Why use parameterized SQL?**
4. **Why enable SQLite foreign keys on each connection?**
5. **Why use a composite uniqueness constraint on player_id and item_id?**

Assigned study

R18: tables, queries, joins, foreign keys and indexes via the linked manual.

Expansion source map: R18, P10, P11, P12. See the source directory for the exact documents and version notes.

19.4 — Transactions, consistency and distributed failure

Learning objective: Preserve invariants when several operations must succeed together.

Original foundation

A transaction groups related database work under defined atomicity, consistency, isolation and durability guarantees. Isolation levels differ; applications must understand which anomalies remain possible. A transfer should not debit one account without the corresponding credit. Concurrency can require retries even when each statement is individually correct. Across multiple services, a local transaction does not automatically create a global one. Partial failure, duplicate messages and stale reads require explicit coordination. Separate durable authoritative data from caches. A cache miss is recoverable; losing the only authoritative record is a different failure. Test the system's invariants under interruption rather than only normal response codes.

Expanded study

Choose the invariant before writing updates

A transfer of ore between two players must preserve total ore, keep both quantities valid and either apply both sides or neither. A transaction groups the debit, credit and replay record into one commit. A database can enforce the local transaction boundary; it cannot automatically include effects in another service or a client display. Record the authority boundary explicitly. [R18]

Make retries part of the transaction

The operations table stores operation ID, canonical payload and result. The transaction first checks for an existing ID. An identical payload returns the stored response; a conflicting payload is rejected. For new work, it verifies players, available quantity and destination capacity, applies both changes, and inserts the replay record before commit. Storing the replay record after a separate commit would create a window in which a retry could repeat the effect.

Understand the chosen isolation mechanism

The SQLite reference uses BEGIN IMMEDIATE to establish its write transaction before the read/check/update sequence. A competing writer may wait or report a busy condition. This simplifies the single-database exercise but is not a claim about PostgreSQL row-level locking or every isolation level. PostgreSQL study should cover the chosen isolation level and its retryable failures using that server's own documentation. [P12, R18]

Inject on both sides of commitment

The test suite raises after debit, before commit and after commit. The first two paths roll back all transaction changes. The final path simulates losing a response after persistence: it must not attempt to reverse a completed transaction merely because the caller did not receive the reply. A restart using the same durable database can still recognize a repeated operation. Replacing the database with a fresh one loses that history and is outside the replay guarantee.

Worked reasoning

Start at (A=20,B=3). A five-ore transfer produces (15,8), preserving total 23. Failure after the debit must leave (20,3) with no operations row. Failure after commit leaves (15,8) plus one operation record. Retrying the same ID and payload returns the original response and remains at (15,8). Retrying the same ID with quantity 6 is rejected without changing inventory.

Runnable example

File: `python/examples/lesson_19_4.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 19.4 --build-dir build
```

```
from python.reference.lab_19_3 import connect_store,seed_inventory
from python.reference.lab_19_4 import transfer,ResponseLost
db=connect_store()
try:
    seed_inventory(db)
    try:
        transfer(db,"op1","A","B","ore",5,"after_commit")
    except ResponseLost:
        print("reply lost; retry same request")
    print(transfer(db,"op1","A","B","ore",5))
    print("total=",db.execute("SELECT SUM(quantity) FROM inventory").fetchone()[0],sep="")
finally:
    db.close()
```

Expected output from the supplied reference in the validated environment:

```
reply lost; retry same request
{'destination_quantity': 8, 'source_quantity': 15}
total=23
```

Practical checkpoint

Implement an inventory transfer with transactions and forced failure between updates. Pass: verify conservation, rollback and duplicate-request handling; document what is not guaranteed across processes or services.

Expanded contract: Python transfer requires distinct known players, valid identifiers and quantity 1..1000000. It conserves quantity, enforces destination capacity, persists at most 1024 replay records, and rejects conflicting ID reuse. ResponseLost after_commit represents committed work with a lost reply, not rollback.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `python/student/lab_19_4.py`. Use `python3 check_lab.py 19.4 --student` after implementation.

Independent transfer

The debit is committed in database X and the credit is sent to service Y, which times out. Is a local SQL transaction enough?

Quiz — answer before opening the key

1. **What does transaction atomicity mean?**
2. **Does a transaction in one database automatically include another service?**
3. **What invariant should an item-transfer test preserve?**
4. **Why insert the replay record in the same transaction as the effect?**
5. **Does a caught after_commit error imply total quantity changed incorrectly?**

Assigned study

R18: Transactions; R16: persistence and failure reasoning.

Expansion source map: R18, R16, P10, P12. See the source directory for the exact documents and version notes.

Module integration checkpoint

Original project brief: Build a local-only client/server message exercise and a small relational game-data service. Test malformed framing, transaction rollback and duplicate requests without exposing a public service.

Build a loopback or in-process protocol harness and a relational inventory operation. State exactly which behavior belongs to transport, parsing and persistence.

1. A four-byte big-endian framing specification, fragmented/coalesced parser checks and a truncated-EOF test.
2. An absolute receive deadline, byte/field bounds and strict versioned schema validation.
3. Primary/foreign/unique/check constraints, parameterized queries and a before/after index plan.
4. Atomic transfer with injected failure, conserved quantity and an operation replay record in the same transaction.
5. A lost-response retry and conflicting-ID case; retain history and state the replay window.

Live review: The first reply is lost after commit, then the same ID arrives with a different payload. Show that the old effect remains and the conflicting request is rejected.

The workbook supplies the additional exercises and five-question module review. Preserve at least five new edge/failure cases beyond copied examples. A mentor reviews the reasoning as well as the outputs.